

Inheritance

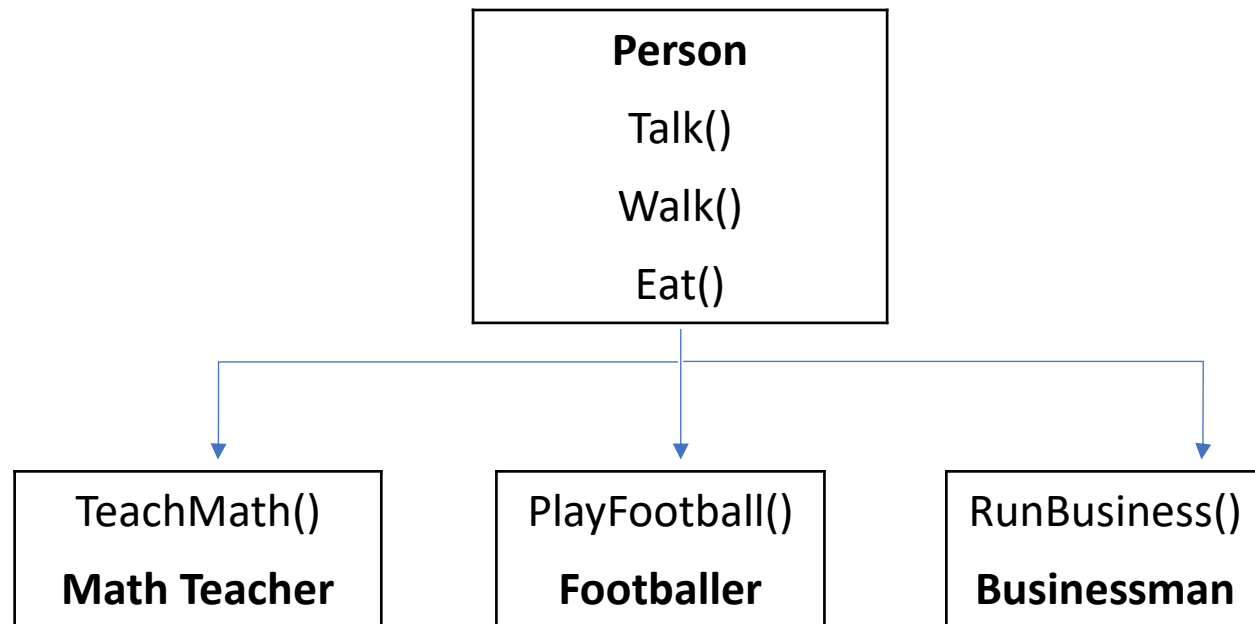
CS1004 – Object
Oriented Programming

Inheritance

- Inheritance is probably most powerful feature of OOP
- Using inheritance we create new classes from existing classes
- Reusability is an important design goal of software engineering
 - write once, use multiple times
 - Saves time and effort
 - Improves program structure and reliability
- Inheritance
 - provides us a mechanism of **software reusability** which is one of the most important principles of software engineering
 - The process of creating **new classes**, called **derived classes**, **from existing classes or base classes** creating a **hierarchy of parent classes and child classes**.
 - The child classes inherit the attributes and behaviour of the parent classes.

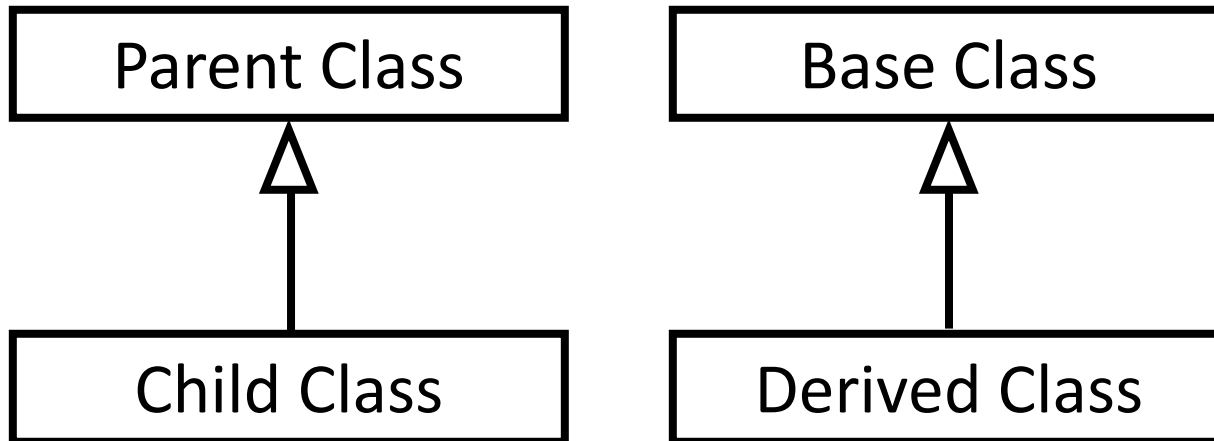
Inheritance in classes

- If a class '**Math Teacher**' inherits from class **Person**, then '**Math Teacher**' contains all the characteristics (information structure and behavior) of class **Person**
- The parent(Person) class is called **base class** and the child class(Math Teacher) is called **derived class**
- Besides inherited characteristics, derived class may have its own unique characteristics, such as 'Teaching Mathematics'
 - or over-ride inherited behaviour.

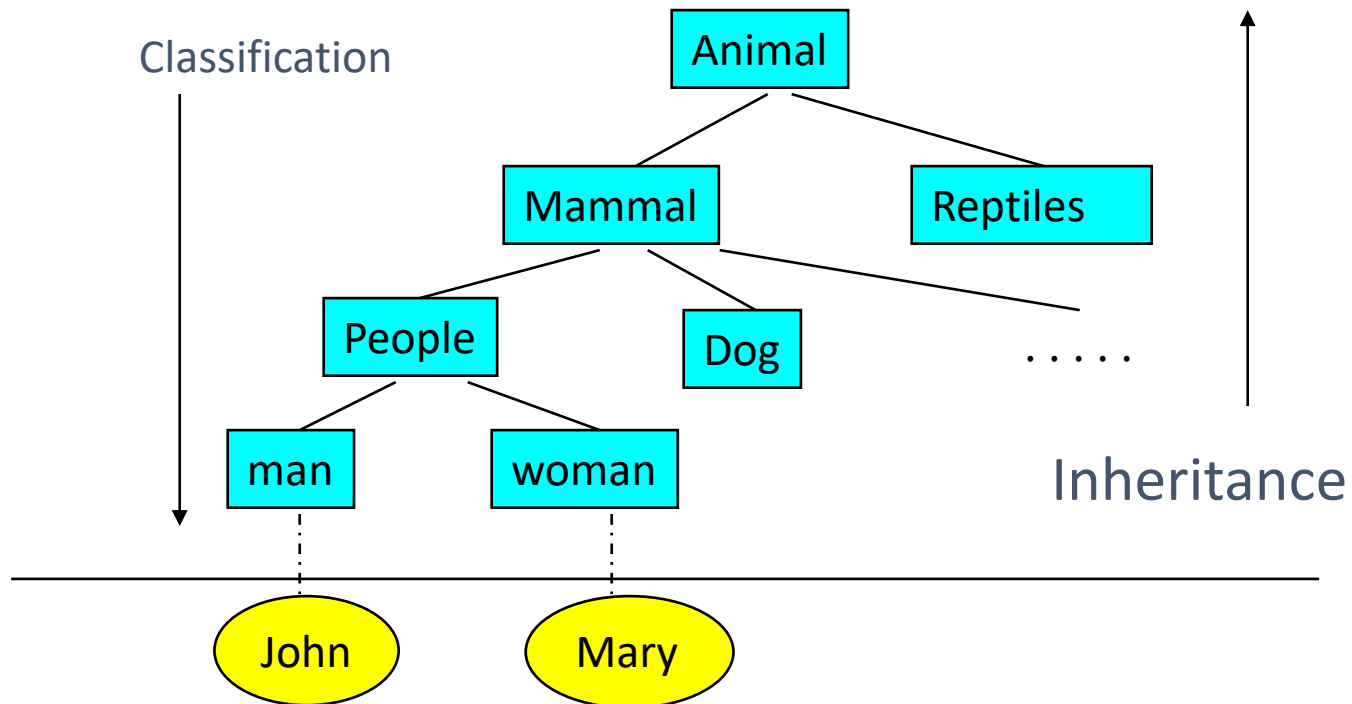


UML Notation

- Child class is derived from Parent class
- Derived Class is derived from Base Class



Animals: Class's hierarchy

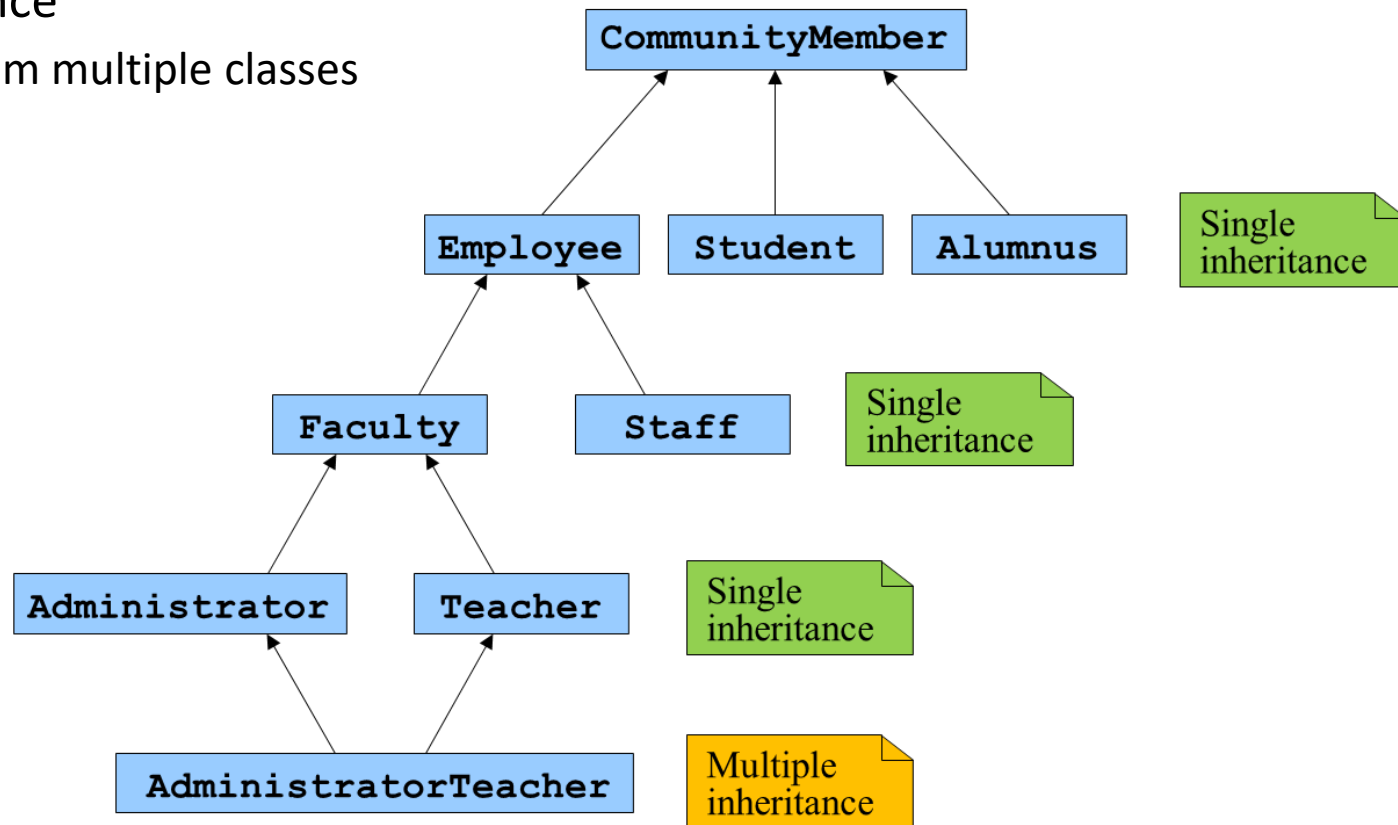


Inheritance Examples

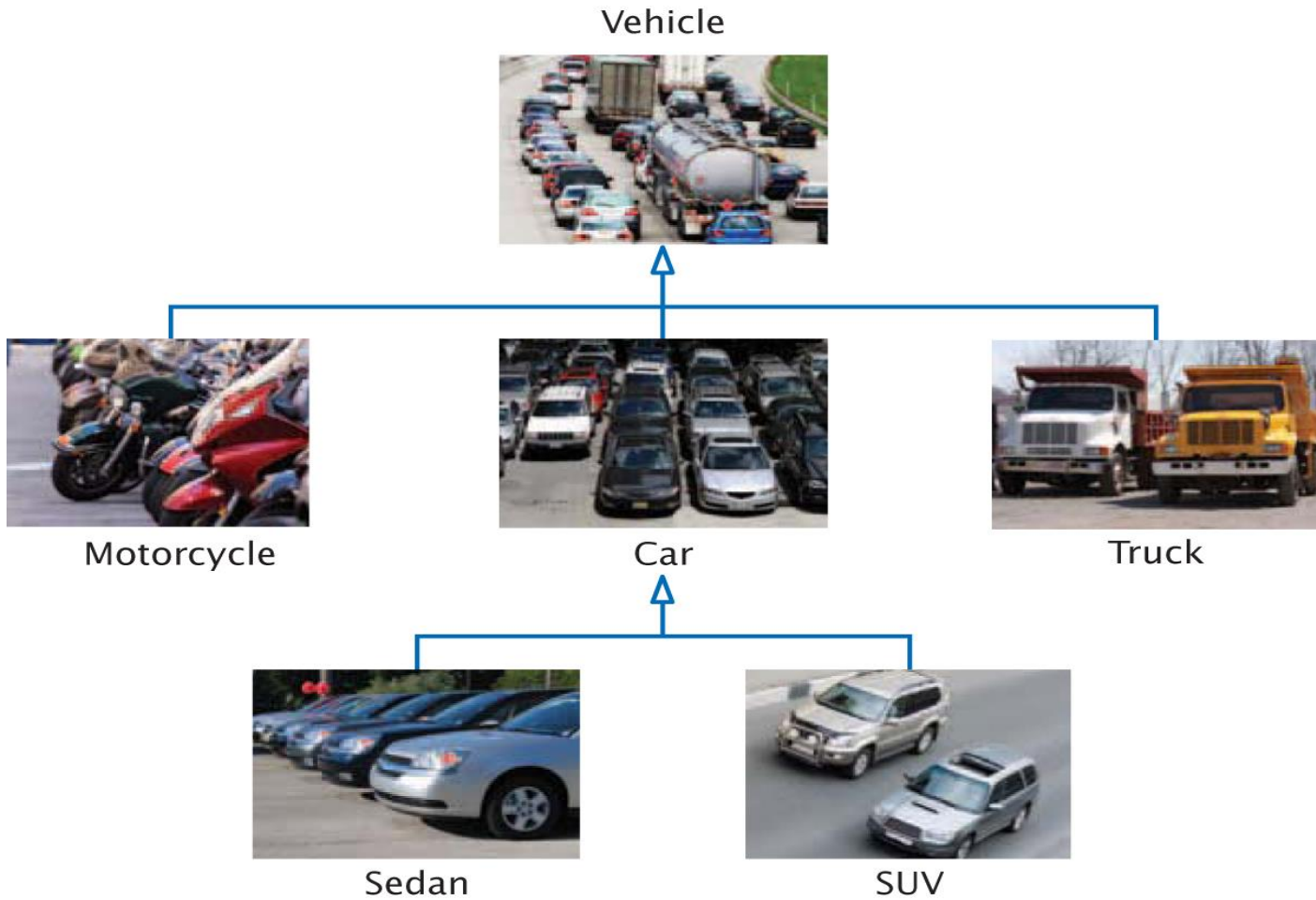
Base Class	Derived Class
Student	GraduateStudent UndergraduateStudent
Shape	Circle Triangle Rectangle
Loan	CarLoan HomeImprovementLoan
Employee	StaffMember FacultyMember
Account	CheckingAccount SavingAccount

Another example: University's community member's hierarchy

- Direct base class
 - Inherited explicitly (one level up hierarchy)
- Indirect base class
 - Inherited two or more levels up hierarchy
- Single inheritance
 - Inherits from one base class
- Multiple inheritance
 - Inheritance from multiple classes



Inheritance Hierarchies



Syntax for **using Public** Inheritance

- Public inheritance creates the “is-a” relationship
 - every rectangle *is a* shape
 - every lion *is an* animal
 - every lawyer *is an* employee
- We say that an object of the derived class “is-a” object of the base class
- **Syntax:**

`class DerivedClassName : access-level BaseClassName`

where

- **access-level** specifies the type of derivation (inheritance)
 - private by default, protected or public

- Most commonly used type of inheritance

```
class Student : public Person {  
    //...  
};
```

Derived class

Base class

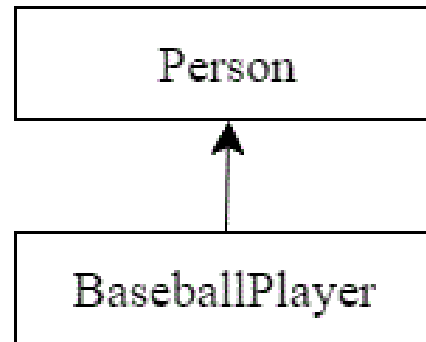
Note: if the type of inheritance is omitted it defaults to ***private***!

Inheritance in C++

- There are three types of inheritance in C++
 - Public
 - Private
 - Protected
- Public inheritance is commonly used form of inheritance.
- Protected and Private inheritance are rarely used.

Example: A Person Class

- Here's a simple class to represent a generic base class named person and a BaseballPlayer class which is derived from the Person Class.
- Every person (regardless of gender, profession, etc...) has a name and age, so those are represented in the Person Class.



Example: A Person Class

- Every person (regardless of gender, profession, etc...) has a name and age, so those are represented here.

```
#include <string>

class Person {
    // In this example, we're making our members public for simplicity
public:
    string m_name;
    int m_age;

    Person(string name = "", int age = 0)
        : m_name( name ), m_age( age )
    {
    }

    string getName() { return m_name; }
    int getAge() { return m_age; }
};
```

Example (cont'd): A Baseball Player

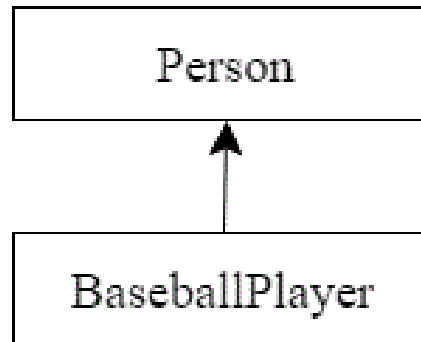
- Baseball players need to contain information that is specific to baseball players -- for example:
- we might want to store a player's *batting average*, and the number of *runs* they've hit.

```
class BaseballPlayer {  
    // In this example, we're making our members public for simplicity  
public:  
    double m_battingAverage;  
    int m_homeRuns;  
  
    BaseballPlayer(double battingAverage = 0.0, int homeRuns =  
0):  
        m_battingAverage(battingAverage), m_homeRuns(homeRuns)  
    {  
    }  
};
```

How to keep track of a baseball player's *name*
and *age*, and while we already have that
information as part of our Person class?

How to keep track of a baseball player's name and age?

- We have following choices for how to add *name* and *age* to BaseballPlayer:
 1. **Option 1:** Add *name* and *age* to the BaseballPlayer class directly as members. This is probably the worst choice, as we're duplicating code that already exists in our Person class.
 2. **Option 2:** Have BaseballPlayer inherit those attributes from Person. Remember that inheritance represents an *is-a* relationship.
 - Is a BaseballPlayer a Person? **Yes**, it is. So inheritance is a good choice here.



Making BaseballPlayer a derived class

- Derived class BaseballPlayer acquires the member functions and variables from Class Person.
- Additionally, BaseballPlayer defines two members of its own: ***m_battingAverage*** and ***m_homeRuns***, since these properties are specific to a BaseballPlayer, not to any Person.

```
class BaseballPlayer : Public Person
{
public:
double m_battingAverage;
int m_homeRuns;

BaseballPlayer(double battingAverage = 0.0, int homeRuns = 0)
    : m_battingAverage(battingAverage), m_homeRuns(homeRuns)
{
}
};
```

Creating Derived class object

- BaseballPlayer objects will have 4 member variables:
- ***m_battingAverage*** and ***m_homeRuns*** from BaseballPlayer, and ***m_name*** and ***m_age*** from Person.

```
int main() {  
    // Create a new BaseballPlayer object  
    BaseballPlayer harry;  
    // Assign it a name (we can do this directly because m_name is public)  
    harry.m_name = "Harry";  
    // Print out the name  
    cout << harry.getName() << endl; // use the getName() function we've acquired  
    from the Person base class  
  
    //this outputs: Harry  
    return 0;  
}
```

Public Inheritance

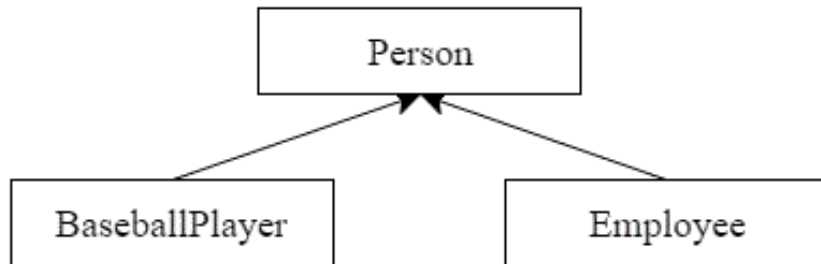
- While implementing the BaseballPlayer class we used the public inheritance:

```
class BaseballPlayer : Public Person
```

- Using **public Inheritance**
 - the public members of the base class (Person) are also public in the derived class (BaseballPlayer)
 - Private members of base class are not accessible from outside of base class, even in the derived class (Information Hiding)
 - private members can be accessed using the member functions of the derived class.
- For example, in our previous example, Person's data members **name** and **age** are accessible by the derived class object.

An Employee Derived Class

- Now we derive a second class named Employee from the Person class.
- An employee “**is-a**” person, so using inheritance is appropriate:



Employee inherits *m_name* and *m_age* from Person (as well as the two access functions), and adds two more member variables and a member function of its own.

An Employee Derived Class

- We derive an Employee class from Person. An employee “is-a” person, so using inheritance is appropriate.
 - Employee class has two data members and one member function.

// Employee publicly inherits from Person

```
class Employee: public Person {  
public:  
    double m_hourlySalary;  
    long m_employeeID;
```

```
    Employee(double hourlySalary = 0.0, int employeeID = 0):  
        m_hourlySalary(hourlySalary), m_employeeID(employeeID)  
{  
}
```

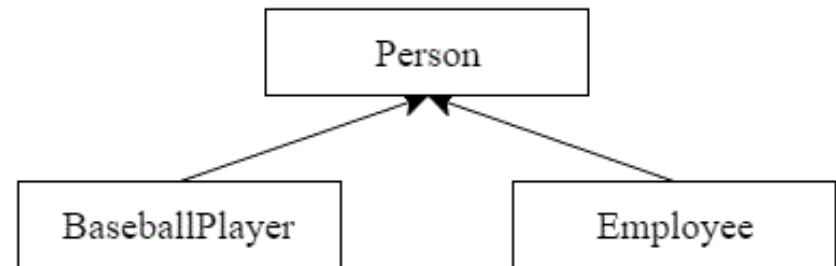
```
void printNameAndSalary() const  
{ //Note that Employee class can access its Base class public members directly.  
    cout << m_name << ": " << m_hourlySalary << endl;  
}  
};
```

Employee inherits `m_name` and `m_age` from Person (as well as the two access functions), and adds two more member variables and a member function of its own.

Creating Derived class Employee object

- Employee objects will have 4 member variables:
- ***m_hourlySalary*** and ***m_EmpID*** from Employee, and ***m_name*** and ***m_age*** from Person.
- The main() outputs name and salary of the employee object.
 - Employee inherits name from Person

```
int main()    {  
    // Create a new employee object  
    Employee emp (1000, 25); //calls constructor with arguments hourllysalary, and empID  
    // Assign it a name (we can do this directly because m_name is public)  
    emp.m_name = "Smith";  
  
    emp.PrintNameandSalary();  
  
    return 0;  
}
```



Modified Base class (Person)

- Now let's make the data members private and member functions public.
- And let's try to access m_name in the derived class Employee

```
#include <string>
class Person {
    Private:
        string m_name;
        int m_age;
    Public:
        Person(string name = "", int age = 0):
            m_name( name ), m_age( age )
        {
        }
        string getName() { return m_name; }
        int getAge() { return m_age; }
};
```

Inheriting from the base class

- We publicly inherit the Employee class from the Person class

```
// Employee publicly inherits from Person
```

```
class Employee: public Person {  
public:  
    double m_hourlySalary;  
    long m_employeeID;  
  
    Employee(double hourlySalary = 0.0, int employeeID = 0)  
        : m_hourlySalary(hourlySalary), m_employeeID(employeeID)  
{ }
```

It will generate an ERROR as **m_name** is a private member of base class and cannot be accessed directly in derived class.

```
void printNameAndSalary() const {
```

```
//Note that Employee class CANNOT access its Base class private members directly.
```

```
    cout << m_name << ": " << m_hourlySalary << endl;  
}
```

```
};
```

Accessing member of base class

However, derived class can access private members of base class via public member functions.

```
void Employee::printNameAndSalary() const
{//derived class CAN access its Base class private members using public member function.
    cout << getname() << ": " << m_hourlySalary <<
endl;
}
```

Driver program to test Employee class

```
int main() {  
    Employee emp;  
    emp.m_name = "Ali"; //ERROR: m_name is private.  
    // Print out the name  
    cout << emp.getName() << endl;  
  
    return 0;  
}
```

'Protected' Access Specifier

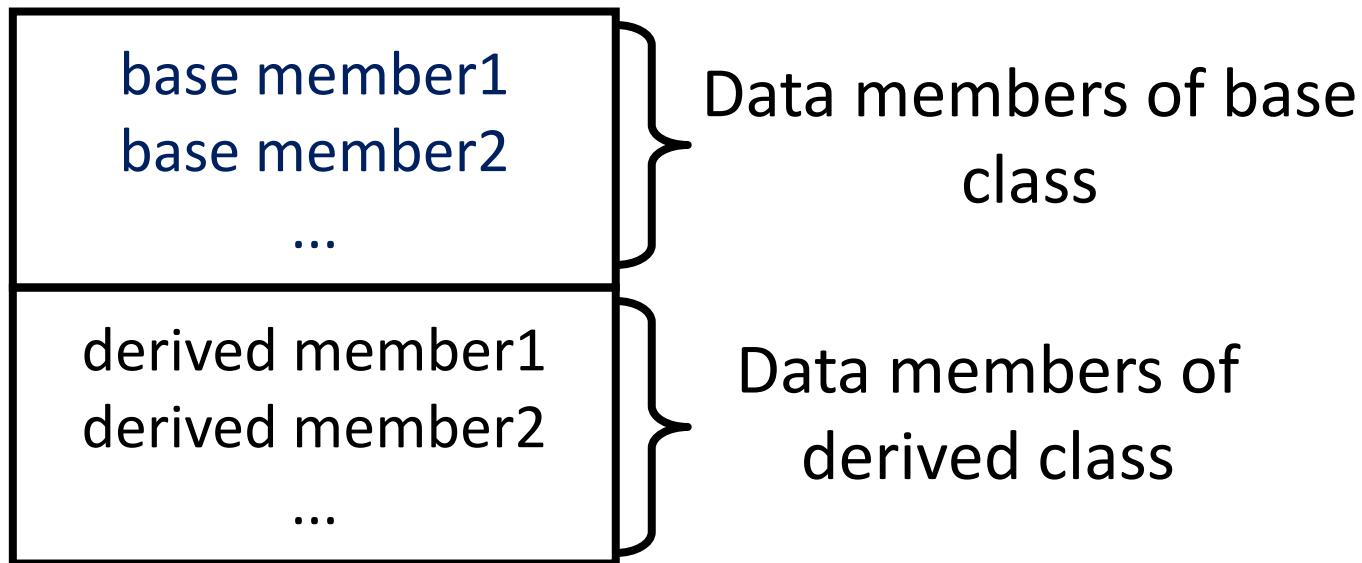
- We have already covered the public and private access specifiers.
- There is a third access specifier '**Protected**' which is only useful in the context of inheritance.
- A member declared as **protected** is accessible by the member functions within its class and any class immediately derived from it.
- It cannot be accessed by the function outside these two classes.

'Protected' Access Specifier: Another Example

```
class person {
private:
    string name;
protected:
    int age;
public:
void setname(string n){name = n; }
Void setage(int a) { age = a;}
    string getname() { return name;}
};
class employee : public person {
public:
void printdetail(){ cout<<"employee name is: "<<getname()<<" age is: "<<a
ge; } // allowed: age a protected member can be accessed directly in the derived class
};
int main() {
    person p;
p.setname("Asif");
p.age = 20; // not allowed: can not access protected members from outside class
p.setage(20);
Employee e;
e.setname("Arif");
e.setage(15); // allowed: can access protected member using base class public member function
e.printdetail(); // allowed: can access protected member using derived class public member functio
n
    return 0; }
```

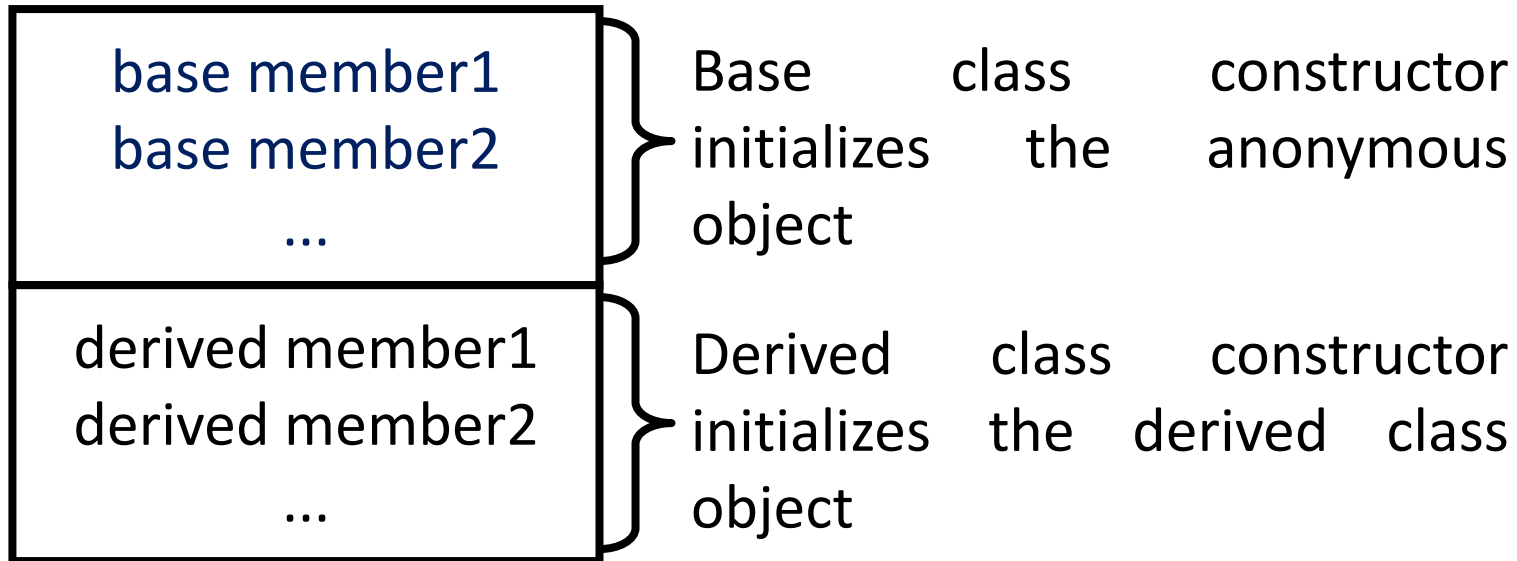
Memory footprint

- Every object of derived class has an anonymous object of base class
- The object of derived class is represented in memory as follows



Constructors

- The anonymous object of base class must be initialized using constructor of base class
- When a derived class object is created the constructor of base class is executed before the constructor of derived class



Constructors in Inheritance

- Constructors :
 - Derived class constructor implementation should call parent class constructor
 - Make call explicit in initialization list
 - Calling order: bottom-up
 - Execution order: top-down

Inheritance With Default Constructors

- Both Parent and child classes have default (no-argument constructors)
- In this case, the subclass constructor automatically and implicitly calls the Base class default constructor.
- When a derived class object is instantiated, constructor of the base class is called, initializing the base class private members.
- After the base class constructor completes execution, it returns control to derived class constructor which initializes the derived class object.

```
class Person {  
public:  
    Person() {  
        cout << "Person Constructor...";  
    }  
};  
  
class employee : public Person {  
public:  
    employee() {  
        cout << "Employee Constructor...";  
    }  
};
```

```
int main() {  
    Employee Emp;  
}
```

Output:

```
Person Constructor...  
Employee Constructor...
```

Calling base class Default Constructors explicitly

- It is also possible that the Employee class constructor explicitly calls the Person class constructor.
- For this purpose **constructor initializer list** can be used to explicitly call the base class constructor.

```
class Person {  
public:  
    Person() {  
        cout << "Person Constructor...";  
    }  
};  
class employee : public Person {  
public:  
    employee():Person() {  
        cout << "Employee Constructor...";  
    }  
};
```

Output:
Person Constructor..
Employee Constructor...

```
int main() {  
    Employee Emp;  
}
```

Inheritance with non-default Constructor

- If default constructor of base class does not exist, then the compiler will try to generate a default constructor for base class and execute it before executing constructor of derived class
- If the user has given only an parameterized constructor for base class, the compiler will not generate default constructor for base class
- In Example below, Person constructor takes argument. But Employee constructor called is assuming the call to the default Person constructor without any arguments.

```
class Person {
Private:
    ...
Public:
    Person(string name, int age):m_name( name ), m_age( age )
    {
    }
};

class Employee: Public Person {
Private:
    ...
Public:
    Employee() //ERROR
    {
    }
};
```

```
int main() {
    Employee Emp; //ERROR
}
```

Inheritance with non-default Constructor

- In case of non-default base class constructor, the derived class constructor will explicitly call the base class constructor using the constructor initializer list.

```
class Person {
Private:
    ...
Public:
    Person(string name, int age):m_name( name ), m_age( age )
    {
    }
};

class Employee: Public Person {
Private:
    ...
Public:
    Employee(string a_name, int a_age):Person(a_name,a_age)
    {
    }
};
```

```
int main() {
    Employee Emp("Sue Smith", 25);
}
```

Inheritance with non-default Constructor having default arguments

- In case of non-default base class constructor, the derived class constructor will explicitly call the base class constructor using the constructor initializer list.

```
class Person {
Private:
    ...
Public:
    Person(string name="", int age=0):m_name( name ), m_age( age )
    {
    }
};

class Employee: Public Person {
Private:
    ...
Public:
    Employee(string a_name="", int a_age=0):Person(a_name,a_age)
    {
    }
};
```

```
int main() {
    Employee Emp1("Sue Smith", 25);
    Employee Emp2;
}
```

Inheritance with non-default Constructor having default arguments

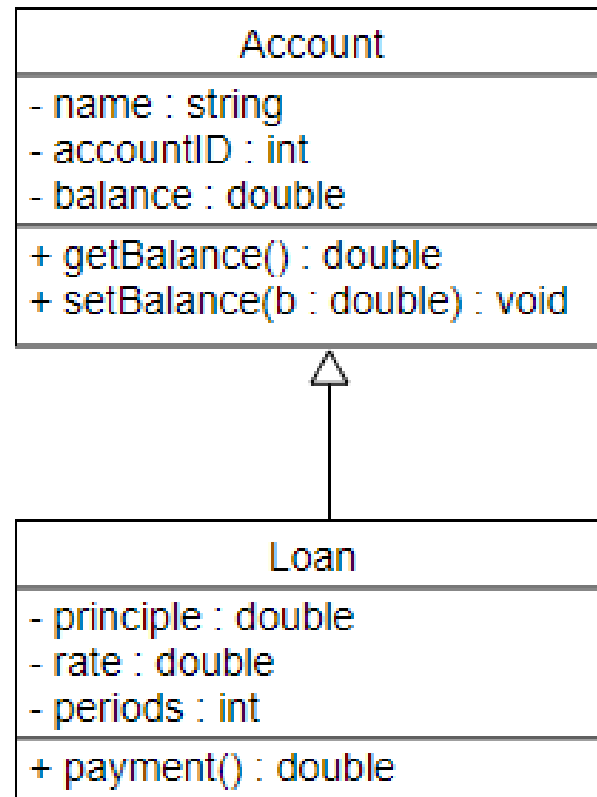
- As long as we create objects of the derived class only (without passing any arguments) then (1) we do not need base class constructor with default arguments (2) we need derived class constructor with default argument.
- In the below example, we create object Emp2, which does not need any arguments, but we still use default arguments.

```
class Person {
Private:
    ...
Public:
    Person(string name, int age):m_name( name ), m_age( age )
    {
    }
};

class Employee: Public Person {
Private:
    ...
Public:
    Employee(string a name="", int a age=0):Person(a_name,a_age)
    {
    }
};
```

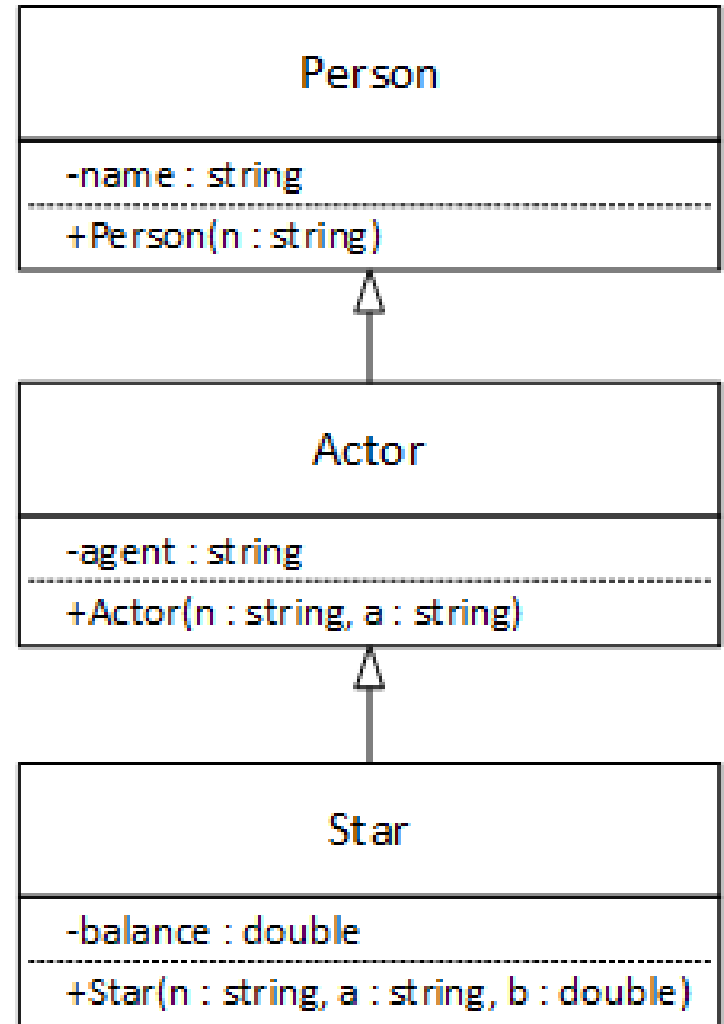
```
int main() {
    Employee Emp2;
}
```

Uml diagram for inheritance



Another Example:

```
class Person {  
  Private:  
    ...  
  Public:  
  
};  
class Actor: Public Person {  
  Private:  
    ...  
  Public:  
  
};  
Class Star: public Actor {  
  Private:  
    ...  
  Public:  
    ...  
};
```



Another Example: Inheritance With Non-Default Constructors

- The child can only perform the initialization of direct base class through *base class initialization list*
- The child can not perform the initialization of an indirect base class through *base class initialization list*

Person Constructor	<code>Person(string a_name) : name(a_name) {}</code>
Actor Constructor	<code>Actor(string name, string a_agent) : Person(name), agent(a_agent) {}</code>
Star Constructor	<code>Star(string name, string agent, double a_balance) : Actor(name, agent), balance(a_balance) {}</code>

Figure 6. Constructor parameter names. The variable names appearing in a constructor's initializer list are not arbitrary. Programmers may choose the constructor parameters' names, but once chosen, *programmers must use the names consistently throughout the function, including the initializer list*. Review the classes in Figure 1 and notice that Person has a member variable called "name" while Actor has a member called "agent." The Actor constructor illustrates the consistent use of variable names:

- The first constructor parameter is called "name," which does not conflict with any Actor member variables ("name" is a Person member). The second constructor parameter is called "a_agent," which distinguishes it from the Actor member "agent." The initializer list uses the same variable names.
- The first element in the initializer list, `Person(name)`, uses the parameter "name;" the second element, `Person(string a_name)`, must use the parameter "a_agent."

I often use single-character variable names in textbook examples to simplify and save space on the page. But, when writing "real" code, it is good practice to choose names that better suggest what the variable does.

Another Example: Inheritance With Non-Default Constructors

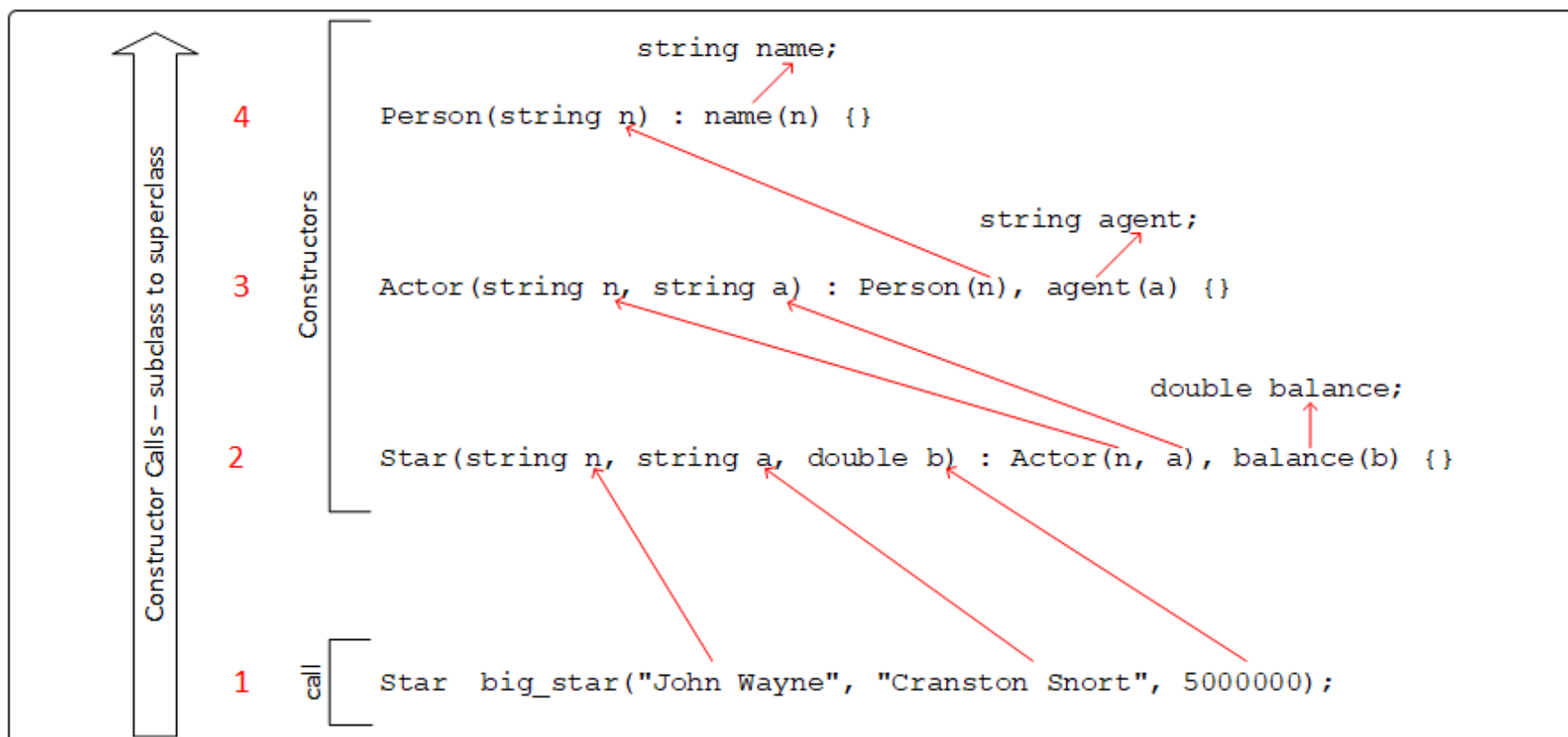


Figure 7. Data flow in chained constructor calls. The arrows indicate the direction that data, which originates in the function call at the bottom (1), flows upwards through the constructors, the initializer lists, and finally into the member variables.

1. The chain begins when the program instantiates an instance of `Star` (`big_star`) and passes three values into the constructor.
2. The `Star` constructor uses one parameter, `b`, to initialize its member variable, `balance`, and passes the values in the remaining two parameters, `n` and `a`, to the `Actor` constructor.
3. The `Actor` constructor uses one parameter, `a`, to initialize its member variable, `agent`, and passes the value in the remaining parameter, `n`, to the `Person` constructor.
4. The `Person` constructor uses the parameter, `n`, to initialize its member variable, `name`.

Order of initialization

- Data member are initialized in order they are declared
 - the execution takes place according to the order of declaration:
- Order in member initializer list is not significant at all

```
class TEST {  
public:  
    int a;  
    int b;  
    int c;  
    TEST();  
};  
TEST::TEST() :b(10), a(b), c(b) { }
```

```
main() {  
    Test t;  
    cout<<" a: "<<t.a<<" b: "<<t.b<<" c: "<<t.c<<endl  
}
```

Output:

```
a will have junk value  
b will have 10  
c will have 10
```

Constructors and Destructors in Base and Derived Classes

- Derived classes can have their own constructors and destructors
- When an object of a derived class is created, the base class's **constructor is executed first**, followed by the derived class's constructor
- When an object of a derived class is destroyed, **its destructor is called first**, then that of the base class

Destructors in Inheritance

- Destructors:
 - No explicit call to base class destructor
 - Called automatically as object departs scope
 - Calling order: bottom-up
 - Execution order: bottom-up

Code example: Constructors and destructors in Inheritance

```
class Base {
public:
    Base () {
        std::cout << "Base::Base() \n";
    }
    ~Base () {
        std::cout << "Base::~~Base() \n";
    }
};

class Derived : public Base {
public:
    Derived () : Base() {
        std::cout << "Derived::Derived() \n";
    }
    ~Derived () {
        std::cout << "Derived::~~Derived() \n";
    }
};

int main() {
    Derived D;
}
```

```
Base::Base() Constructor
Derived::Derived() Constructor
Derived::~~Derived() Destructor
Base::~~Base() Destructor
```

Dynamic Allocation in Base and Derived class

```
class Base {
public:
    Base() { x = new int;
        cout<<"Allocation in Base:"<<endl; }
    ~Base() { delete x;
        cout<<"Deallocation in Base:"<<endl; }
private:
    int* x;
};

class Der1 : public Base {
public:
    Der1() { y = new int;
        cout<<"Allocation in Derived:"<<endl; }
    ~Der1() { delete y;
        cout<<"Deallocation in Derived:"<<endl; }
private:
    int* y;
};

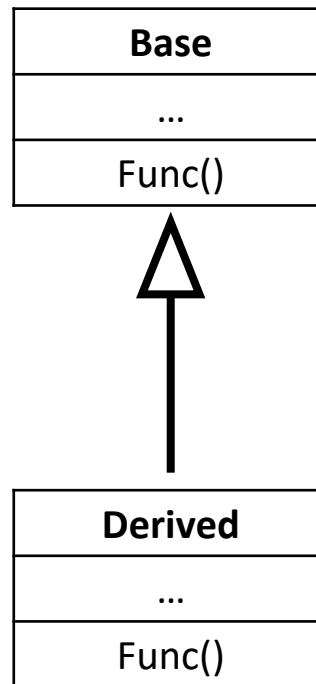
void main() {
    Der1* D = new Der1;
    delete D;
}
```

- Destructors are explicitly called for dynamically allocated objects using delete:

```
Allocation in Base:
Allocation in Derived:
Deallocation in Derived:
Deallocation in Base:
```

Overriding member functions of base class

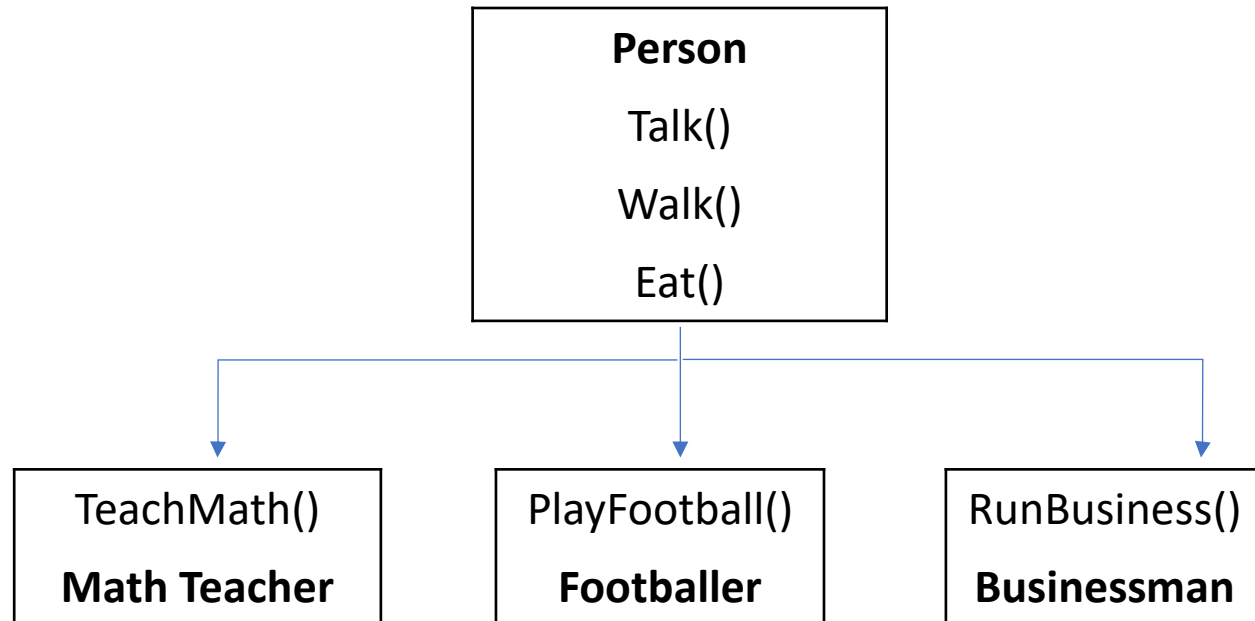
- Sometimes a derived class needs to have the same function name as that in a base class but with different functionality.
- This is achieved by function overriding.
- Overriding a function is simple: just reimplement the function with the same name and arguments in the derived class.



Overriding example

- Derive class can override member function of base class such that the working of function is similar to former implementation

```
class Person {  
public:  
    void Walk();  
};  
class ParalyzedPerson : public Person {  
public:  
    void Walk();  
};
```



Calling inherited functions and overriding(redefining) behavior

- By default, derived classes **inherit all of the behaviors** defined in a base class.
- When a member function is called with a **derived class** object, the compiler first looks to see if that member exists in the **derived class**.
- If not, it begins walking up the inheritance chain and checking whether the member has been defined in any of the **parent classes**. It uses the first one it finds.

Calling inherited functions and overriding behavior

Derived class inherits member function identity() from base class and use it.

```
class Base
{
public:
    Base(int value)
        : m_value(value)
    {
    }
    void identify() { std::cout << "I am a Base\n"; }
    int m_value;
};
```

```
int main()
{
    Base base(5);
    base.identify();

    Derived derived(7);
    derived.identify();

    return 0;
}
```

This prints

I am a Base

I am a Base

```
class Derived: public Base
{
public:
    Derived(int value)
        : Base(value)
    {
    }
};
```

Calling inherited functions and overriding behavior

- When `derived.identify()` is called, the compiler looks to see if function `identify()` has been defined in the Derived class. It hasn't.
- Then it starts looking in the inherited classes (which in this case is Base). `Base has defined an identify() function`, so it uses that one.
- In other words, `Base::identify()` was used because `Derived::identify()` doesn't exist.

Redefining (overriding) behavior

- However, if we had defined `Derived::identify()` in the Derived class, it would have been used instead.
- This means that we can make functions work differently with our derived classes by **redefining (overriding)** them in the derived class!
- Below **Derived class redefines** the `identity()` member function.

```
class Derived: public Base
{
public:
    Derived(int value)
        : Base(value)
    {
    }
    int getValue() { return m_value; }

    // Here's our modified function
    void identify() {
        cout << "I am a Derived";
    }
};
```

```
int main()
{
    Base base(5);
    base.identify();

    Derived derived(7);
    derived.identify(); //overrides identity()

    return 0;
}
```

Output:

I am a Base

I am a Derived

Overriding a function: A simple Example

```
class Base {
public:
void PrintNum() {
cout << 1 << endl ;
}
};
class Derived : public Base {
public:
// Override
void PrintNum() {
cout << 2 << endl ;
}
} ;
Base b;
b.PrintNum() ; // Prints 1
Derived d ;
d.PrintNum() ; // Prints 2
```

Class exercise:

- Create a uml diagram for the following and code the program:
- There is a student base class. It has the following:
 - Attributes: Name, age, RollNumber, CGPA
 - Behavior: ShowCGPA()
- Following PGStudent class is derived from the student class.
 - PGStudent
 - Attributes: string thesistitle, string thesissupervisorname
 - Methods: showthesistitle()
- Following instructor class is derived from the PGStudent class.
 - Attributes: float salary, string depttname
 - Methods: showdetails()

Adding to existing functionality

- Note that `Derived::identify()` completely hides `Base::identify()`, in the previous example
- Sometimes we don't want to completely replace a **base class function**, but instead want to add additional functionality to it.
- It is possible to have our derived function call the base version of the function of the same name (in order to reuse code) and then add additional functionality to it.
- We redefine `Derived::identify()` so it first calls `Base::identify()` and then does its own additional stuff.
- To have a derived function call a base function of the same name, simply do a normal function call, but prefix the function with the **scope qualifier** (the name of the base class and two colons).

Adding to existing functionality

- **Caution:** Calling function `identify()` without a scope resolution qualifier would default to the `identify()` in the current class, which would be `Derived::identify()`.
- This would cause `Derived::identify()` to call itself, which would lead to an infinite loop!

```
class Derived: public Base
{
public:
    Derived(int value)
        : Base(value)
    {
    }
    int getValue() { return m_value; }
```

// Here's our modified function

```
void identify() {
Base::identify(); // call Base::identify first
cout << "I am a Derived\n"; //then derived
}
};
```

```
int main()
{
    Base base(5);
    base.identify();

    Derived derived(7);
    derived.identify();

    return 0;
}
```

Output:

I am a Base

I am a Base

I am a Derived

Overloading vs Overriding (Redefining)

- Do not mix overriding and overloading. These are two different concepts

- **Overloading:**

- Allow to use **same name** for member functions with different arguments
- Overloading is done within the scope of one class
- In below example, member function set() is overloaded.

- **Example:**

```
class A {  
    public:  
    void set (int a) { _a = a; }  
    void set (double f) { _f = f; }  
    void set (int a, double f) { _a = a; _f = f; }  
    private:  
    double _f; int _a;  
};
```

- **Overriding (redefining):**

- Allow to **specialize the behavior** of an existing method by providing a different implementation in the derived classes
- Redefining function is only possible with inheritance

Overloading vs overriding

- However, overriding `show(int x)` in the derived class hides the remaining methods with the same name of the base class.

```
class Base {  
public:  
    void show() { cout<<"base::show()"<<endl ; }  
    void show(int x) { cout<<"base::show(int x)"<<endl ; }  
};  
class Derived: public Base {  
public:  
    void show(int x) { cout<<"derived::show(int x)"<<endl ; }  
};  
int main() {  
    Derived d;  
    d.show(4); //OKAY: calls derived::show(int)  
}
```

Overloading vs overriding

- However, overriding `show(int x)` in the derived class hides the remaining methods with the same name of the base class.

```
class Base {  
public:  
    void show() { cout<<"base::show()"<<endl ; }  
    void show(int x) { cout<<"base::show(int x)"<<endl ; }  
};  
class Derived: public Base {  
public:  
    void show(int x) { cout<<"derived::show(int x)"<<endl ; }  
};  
int main() {  
    Derived d;  
    d.show(); //error: no matching function for call to Derived::show()  
}
```

Overloading vs overriding

- We can introduce the base class functions with the same name using `using`:

```
class Base {
public:
    void show() { cout<<"base::show()"<<endl ; }
    void show(int x) { cout<<"base::show(int x)"<<endl ; }
};

class Derived: public Base {
public:
    using Base::show; //introducing base class show() methods
    void show(int x) { cout<<"derived::show(int x)"<<endl ; }
};

int main() {
    Derived d;
    d.show(4); // calls derived::show(int)
    d.show(); // calls base::show()
}
```

- Another example

```
class A {  
public:  
    int ia1, ia2;  
    void fa1();  
    int fa2(int);  
};  
class B: public A {  
public:  
    float ia1;  
    float fa1(float);  
};
```

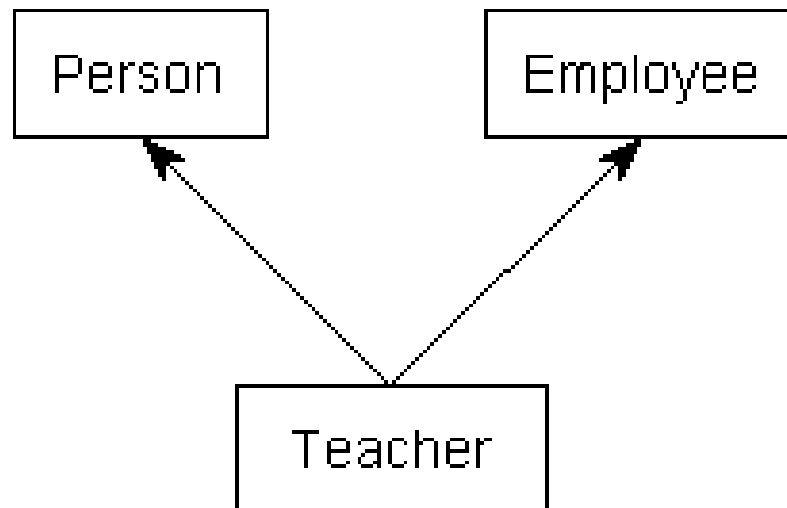
- example

```
class A{
public:
    int ia1, ia2;
    void fa1();
    int fa2(int);
};
class B: public A{
public:
    float ia1;           // overrides ia1
    float fa1(float);     // overrides fa1
};
int main()
{
    B b;
    int j = b.fa2(1);      // A::fa2
    b.ia1 = 4;             // B::ia1
    b.ia2 = 3;             // A::ia2 if ia2 is public in A
    float y = b.fa1(3.14); // B::fa1
    b.fa1();               // ERROR! fa1 func. in B hides function of A
    b.A::fa1();            // OK
    b.A::ia1 = 1;          // OK if ia1 is public in A
    return 0;
}
```

Multiple Inheritance

- Multiple inheritance enables a derived class to inherit members from more than one base classes.
- Let's say we want to write a program to keep track of a number of **teachers**.
- A teacher is a **person**.
- However, a teacher is also an **employee**.
- Multiple inheritance can be used to create a **Teacher class** that inherits properties from both **Person and Employee**.
- To use multiple inheritance, simply specify each base class (just like in single inheritance), separated by a comma. For example:

```
class Teacher: public Person, public Employee
```



Multiple Inheritance Example: Teacher class

```
class Person {  
• private:  
    string m_name;  
    int m_age;  
public:  
    Person(string name, int age)  
        : m_name(name), m_age(age)  
    {  
    }  
    string getName() { return m_name; }  
    int getAge() { return m_age; }  
};
```

```
class Employee {  
private:  
    string m_employer;  
    double m_wage;  
public:  
    Employee(string employer, double wage)  
        : m_employer(employer), m_wage(wage)  
    {  
    }  
    string getEmployer() { return m_employer; }  
    double getWage() { return m_wage; }  
};
```

```
class Teacher: public Person, public Employee {  
private:  
    int m_teachesGrade; //Data member specific to Teacher class  
public:  
    Teacher(string name, int age, string employer, double wage, int teachesGrade)  
        : Person(name, age), Employee(employer, wage), m_teachesGrade(teachesGrade)  
    {  
    }  
};
```

Multiple Inheritance Example: Teacher class

```
class Teacher: public Person, public Employee {  
private:  
    int m_teachesGrade; //Data member specific to Teacher class  
public:  
Teacher(string name, int age, string employer, double wage, int teachesGrade)  
    : Person(name, age), Employee(employer, wage), m_teachesGrade(teachesGrade)  
    {  
    }  
};
```

```
int main() {  
  
Teacher teacher("Asif", 40, "city School", 50000, 16);  
cout<<teacher.getWage()<<endl;  
cout<<teacher.getName()<<endl;  
}
```

Another Simple Example of Multiple Inheritance

```
//simple example showing multiple inheritance
class base1 {
public:
    void funbase1(void) { cout<<"funbase1"; }
};

class base2 {
public:
    void funbase2(void) { cout<<"funbase2"; }
};

class derived:public base1, public base2 {
public:
    void funderived(void) { cout<<"funderived";}
};

void main(void) {
    derived der; //der inherits functionalities from both base classes
    der.funbase1();
    der.funbase2();
    der.funderived();
}
```

```
funbase1
funbase2
funderived
```

Constructor & Destructor in Multiple inheritance

```
class baseClass1 {
public:
    baseClass1() {
        cout<<"I am baseClass1 constructor"<<endl;
    }

    ~baseClass1() {
        cout<<"I am baseClass1 destructor"<<endl;
    }
};

class baseClass2 {
public:
    baseClass2() {
        cout<<"I am baseClass2 constructor"<<endl;
    }

    ~baseClass2() {
        cout<<"I am baseClass2 destructor"<<endl;
    }
};
```

```
class derivedClass: public baseClass1, public
baseClass2 {
public:
    derivedClass() {
        cout<<"I am derivedClass
constructor"<<endl;
    }

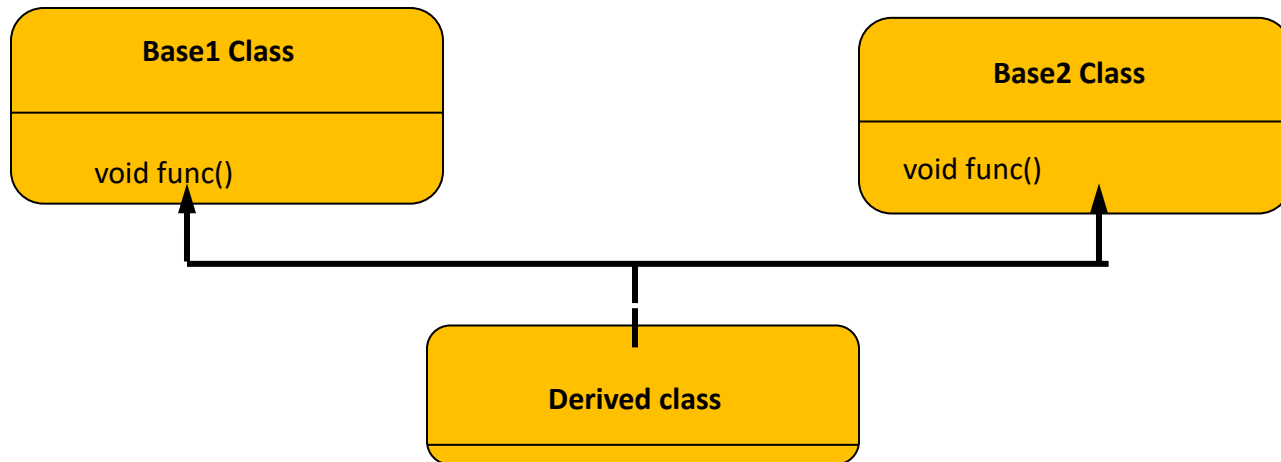
    ~derivedClass() {
        cout<<"I am derivedClass destructor"<<endl;
    }
};

int main() {
    derivedClass D;
    return 0;
}
```

```
I am baseClass1 constructor
I am baseClass2 constructor
I am derivedClass constructor
I am derivedClass destructor
I am baseClass2 destructor
I am baseClass1 destructor
```

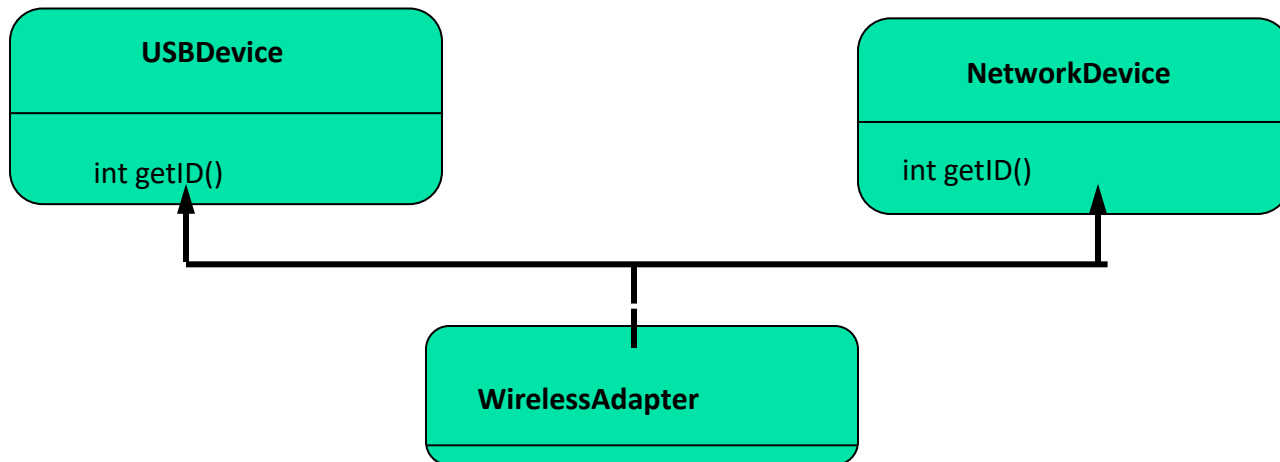
Inheritance Ambiguity in C++ In multiple inheritances

- Multiple inheritance introduces a lot of issues that can increase the complexity of programs and make maintenance difficult.
- When one class is derived from two or more base classes then there may be a possibility that the base classes have functions with the same name.



Problems with Multiple Inheritance

- Let's take a look at some of these situations.
- First, ambiguity can result when multiple base classes contain a function with the **same name**.
- Assume that we are creating a **class WirelessAdapter**. This class contains functionality of a **networking device** so it is derived from the **NetworkDevice class**
- It also contains some functionalities of a **USB** so it is also derived from **USBDevice class**.
- It means **WirelessAdapter** is derived from two base classes, i.e., **NetworkDevice** and **USBDevice**.



Example: Problems with Multiple Inheritance

- The below code will not compile. Because the **netGear** object contains two copies of `getID()` function.

```
class USBDevice {
private:
int m_id;
public:
    USBDevice(long id)
        : m_id(id)
    { }
    int getID() { return m_id; }
};
```

```
class NetworkDevice {
private:
int m_id;
public:
    NetworkDevice(long id)
        : m_id(id)
    { }
    int getID() { return m_id; }
};
```

```
class WirelessAdapter: public USBDevice, public NetworkDevice
{ //inherits two copies of getID().
public:
    WirelessAdapter(int usbId, int networkId)
        : USBDevice(usbId), NetworkDevice(networkId)
    {
    }
};
```

```
int main() {
    WirelessAdapter netGear(54, 18);
    cout << netGear.getID();
    // Which getID() do we call?
}
```

Problems with Multiple Inheritance

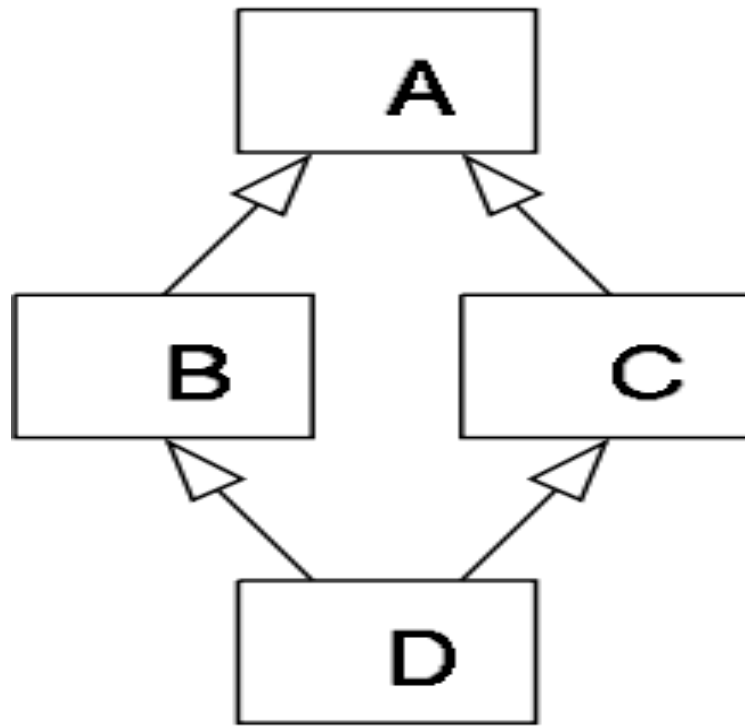
- When `netGear.getID()` is compiled, the compiler looks to see if `WirelessAdapter` contains a function named `getID()`. It doesn't.
- The compiler then looks to see if any of the base classes have a function named `getID()`.
- The problem is that `netGear` actually contains **two `getID()` functions**: one inherited from `USBDevice`, and one inherited from `NetworkDevice`.
- Consequently, this **function call is ambiguous**, and we receive a compiler error
- **Solution:**
- To work around, we can explicitly specify which version we meant to call by using scope resolution operator. For example

```
cout << netGear.NetworkDevice::getID(); //displays 18
```

```
cout << netGear.USBDevice::getID(); //displays 54
```

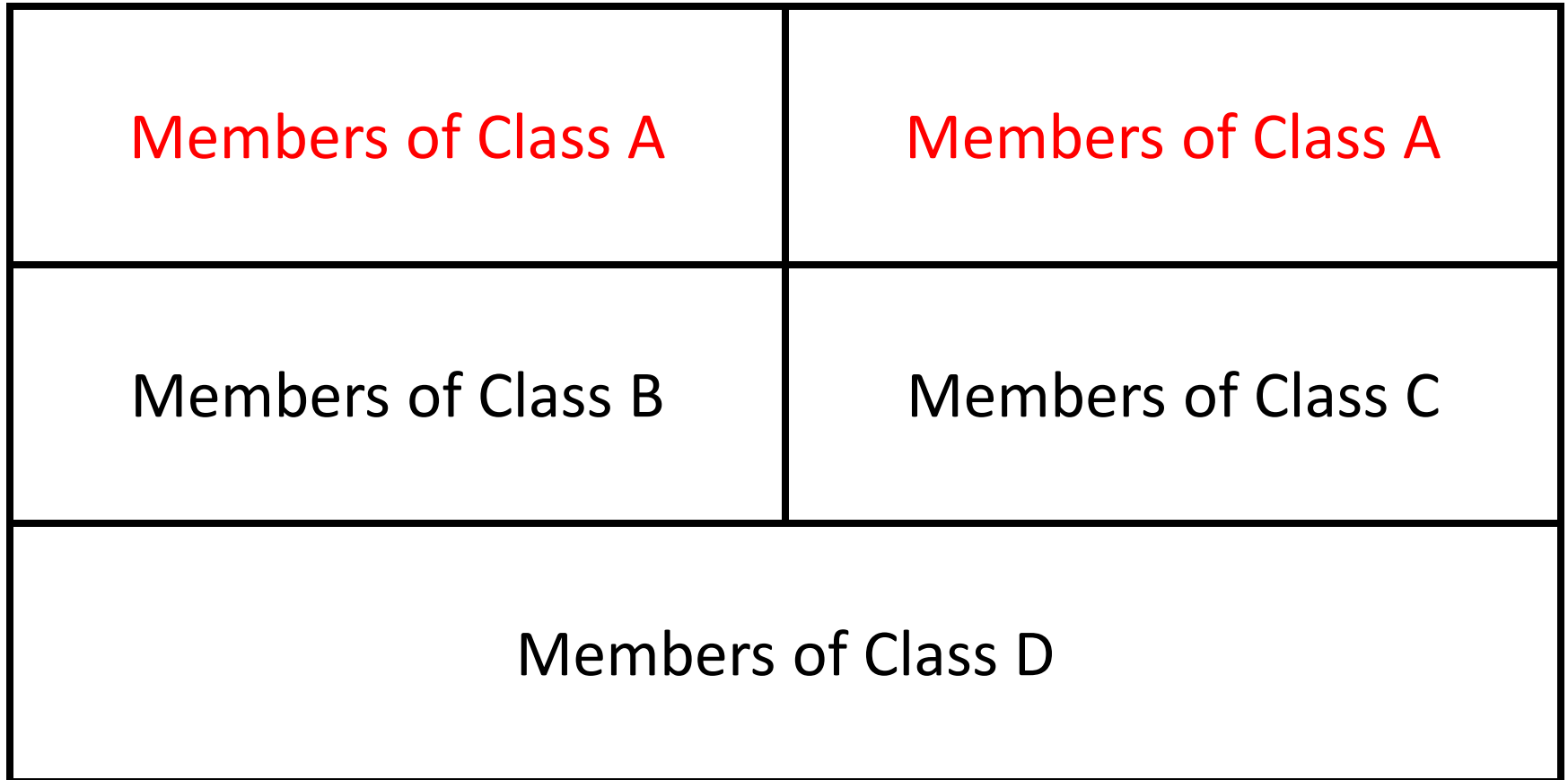
Diamond Problem

- Another problem that arises from multiple inheritance is the **Diamond Problem**.
- if two classes B and C inherit from a class A, and a class D inherits from both B and C, then the derived class D will contain two copies of A's member variables: one via B, and one via C.



Memory View of multiple inheritance

- The derived class D will contain two copies of A's member variables: one via B, and one via C.



Diamond Problem: Example

```
class A { public: void Foo() {} };  
class B : public A {};  
class C : public A {};  
class D : public B, public C {};
```

```
int main() {  
    D d;  
    d.Foo();  
}
```

is this B's Foo() or C's Foo() ??

Line	Message
	=== Build file: "no target" in "no project" (compiler: unknown) ===
..	In function 'int main()':
.. 12	error: request for member 'Foo' is ambiguous
.. 5	note: candidates are: 'void A::Foo()'
.. 5	note: 'void A::Foo()'
	=== Build failed: 1 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===

Solution: Virtual Inheritance

- **Virtual inheritance** is a C++ technique that ensures only one copy of a base class's member variables are inherited by grandchild derived classes.
- Its objective is to allow efficient use of memory and elimination of duplicate state spaces when designing inheritance hierarchies that share a common base class.
- if classes B and C inherit virtually from class A, then objects of class D will contain only one set of the **members** from class A.

Solution (virtual inheritance)

```
class A { public: void Foo() {} }  
class B : public virtual A {}  
class C : public virtual A {}  
class D : public B, public C {}
```

class B: virtual A means, that any class inherited from B is now responsible for creating A by itself, since B isn't going to do it automatically.

```
D d;  
d.Foo(); // no longer ambiguous
```

Memory View after using Virtual Inheritance

Members of Class A	
Members of Class B	Members of Class C
Members of Class D	

Diamond Problem: Another Example

- **B and C inherit A virtually.**
- **This ensures that only one instance of A exists in D, rather than two separate instances from B and C.**
- Since A is virtually inherited, D must explicitly call its constructor:

```
class A {
    string msg;
public:
    A(string x): msg(x) {}
    void test(){ cout<<"hello from A: "<<_msg <<"\n"; }
};

// B,C inherit A virtually
class B: virtual public A    { public: B():A("b"){}}    };
class C: virtual public A    { public: C():A("c"){}}    };

// since B,C inherit A virtually, A must be constructed in D class

// B() and C() constructors can be omitted
class D: public B, public C {
public: D():A("d_a"),B(),C(){}    };

int main(){
    D d;
    d.test();    // prints: "hello from A: d_a"
}
```